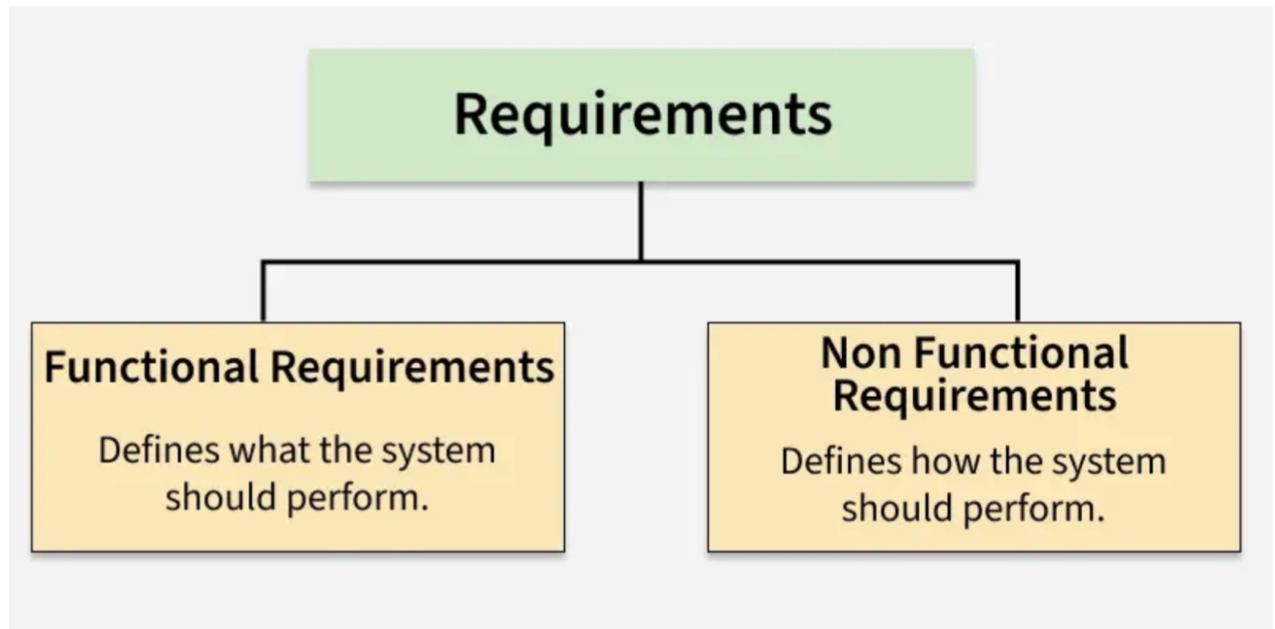


Requirements analysis is an essential process in software development. It helps to determine whether a system or project will meet its objectives and achieve success.

To make this analysis effective, requirements are generally divided into two categories:



What are Functional Requirements?

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it should interact with users or other systems.

- Focus on system behavior and functionality.
- Represent the features that can be directly observed and tested in the final product.
- Common examples include user authentication, data processing, search, payment handling, and report generation.

Sample Questions

- What features should the system include?
- What edge cases must be considered in the design?

What are Non-Functional Requirements?

Non-functional requirements (NFRs) define **how a system should operate**, focusing on performance, reliability, and user experience rather than specific features. They ensure the system is **efficient, secure, and maintainable** over time.

- **Performance** – speed and responsiveness
- **Security** – protection against unauthorized access
- **Usability** – ease of use
- **Reliability** – system stability and availability
- **Scalability** – ability to handle growth

- **Maintainability** – ease of updates and fixes
- **Portability** – ability to run in different environments

Sample Questions

- How fast should the system respond to user actions?
- How secure should it be against unauthorized access?
- How available and reliable should the system be?
- Extended Requirements.

Examples of Functional and Non-functional Requirements

Let's consider a couple of examples to illustrate both types of requirements:

1. Online Banking System

Functional Requirements:

- Users should be able to log in with their username and password.
- Users should be able to check their account balance.
- Users should receive notifications after making a transaction.

Non-functional Requirements:

- The system should respond to user actions in less than 2 seconds.
- All transactions must be encrypted and comply with industry security standards.
- The system should be able to handle 100 million users with minimal downtime.

2. Food Delivery App

Functional Requirements

- Users can browse the menu and place an order.
- Users can make payments and track their orders in real time.

Non-functional Requirements:

- The app should load the restaurant menu in under 1 second.
- The system should support up to 50,000 concurrent orders during peak hours.
- The app should be easy to use for first-time users, with an intuitive interface.

Differences between Functional Requirements and Non-Functional Requirements:

Definition

- Functional requirements define what the system should do, the exact features, tasks, or operations.
- Non-functional requirements define how the system should perform, the qualities or attributes like speed, security, or usability.
- Functional Requirements focuses on the behavior and features of the system.
- Non-Functional Requirements focuses on the performance, usability, and overall quality of the system.

Scope

- Functional Requirements defines the actions and operations the system must support.
- Non functional Requirements defines constraints or conditions under which those actions should occur.

Measurement

- Functional requirements are easily measured by verifying outputs or results.
- Non-functional requirements are harder to measure, often validated against benchmarks, metrics, or SLAs.

Impact on Development

- Functional requirements drive the core design and features of the system.
- Non-functional requirements influence the system architecture and performance optimization.

User Perspective

- Functional requirements are directly visible to users and tied to business needs.
- Non-functional requirements shape the user experience by ensuring efficiency, reliability, and smooth operation.

Documentation

- Functional requirements are documented in use cases, user stories, or functional specifications.
- Non-functional requirements are captured in performance criteria, technical specs, or design constraints.

Evaluation

- Functional requirements are validated through functional testing (unit, integration, or acceptance tests).
- Non-functional requirements are verified via performance, security, and usability testing.

Dependency

- Functional requirements define what must be built to meet user needs.
- Non-functional requirements define how well the system must operate once built.

Examples

- Functional requirements: login authentication, data input/output, transaction processing.
- Non-functional requirements: system scalability, security, response time, reliability, maintainability.

Importance of Balancing Both Functional and Non-Functional Requirements

Balancing functional and non-functional requirements ensures a system is both useful and reliable. Key benefits include:

- **Improves User Experience:** Functional features alone may work but feel slow or hard to use. Non-functional aspects like performance, usability, and availability shape how users interact with the system.
- **Enhances System Performance:** Scalability, reliability, and security ensure the system performs effectively under real-world conditions.
- **Prevents Bottlenecks and Failures:** Proper attention to non-functional needs reduces outages, security breaches, and downtime.
- **Reduces Long-Term Costs:** Addressing non-functional requirements early avoids expensive redesign, rework, or performance fixes later.
- **Supports System Evolution:** Maintainability and extensibility allow the system to scale and adapt smoothly as business and user needs grow.

Common Challenges in Defining these Requirements

Defining functional and non-functional requirements can be complex due to several challenges:

- **Ambiguity in Requirements:** Vague or incomplete requirements make it difficult to define what the system must do (functional) and how it should perform (non-functional).
- **Changing Requirements:** Evolving business goals, market trends, or user expectations can shift requirements, making it harder to maintain stable design.
- **Difficulty in Prioritization:** Functional requirements often take precedence, while critical non-functional aspects like security or scalability may be overlooked.
- **Measuring Non-Functional Requirements:** Functional needs are easier to test; non-functional attributes like usability, scalability, or reliability are harder to quantify and validate.
- **Overlapping or Conflicting Requirements:** Some requirements may conflict, such as stronger security potentially impacting system performance, requiring careful trade-offs.

How to Gather Functional and Non-functional Requirements

Gathering requirements involves multiple approaches and collaboration between the development team, stakeholders, and end-users:

1. Functional Requirements:

- **Interviews:** Talk to stakeholders or users to understand their needs.
- **Surveys:** Distribute questionnaires to gather input from a larger audience.
- **Workshops:** Host sessions to brainstorm features and gather feedback.

2. Non-functional Requirements:

- **Performance Benchmarks:** Consult with IT teams to set expectations for performance and load.
- **Security Standards:** Consult with security experts to define the best practices for data protection.
- **Usability Testing:** Test the system to find areas where users might struggle and refine the interface.